

Finlay Fordham (730038662) and Angelo Thind (730041272) Software

Development CA:

Code implementation Design:

Class and attribute Explanations and Justifications:

Card:

The most basic unit which would be processed most often in this game is a card. Although it would be feasible to simply use an integer type to represent this unit, for the sake of encapsulation, extensibility and readability we have decided to create a Card class instead. The Card stores a private attribute "value", there is a getter for this attribute to comply with the encapsulation paradigm. This attribute is referred to in the CA specification as denomination. Card has the method, getValue() this is needed to get the denomination (number) on the card.

CardCollections:

Since we found that the classes of Pack and Hand could both be classified as a collection of cards and they both require the addition and discarding of cards in the same way (doesn't require an input on either) we decided to create an interface for these classes to implement. This creates a level of abstraction which assists in understanding the problem. It contains two methods, addCard and removeCard which are both to be implemented by Pack and Hand in their own respective ways.

Pack:

This entity is the pack which will be dealt to the players and decks. Its addCard and removeCard methods are simple, when adding it will add to the end of a list and when removing it will remove from the end as well. This is to mimic the adding and dealing from a pack of cards. Here we treat the last item of a list as the top of a stack of cards, thus we remove from the last item of the list. Pack contains an attribute pack which is simply a list of Card objects to keep track of dealing.

Deck:

Deck is very similar to Pack class. However, there are a couple notable differences. Firstly, the addition of a deckId attribute, this is to be able to identify what a player will draw from, or discard to. Secondly, the removeCard method logic is slightly different than that of the removeCard method in Pack class. This is because when drawing from a Deck, the player draws from the top of the deck. For our implementation that would mean drawing from the last item of the list. For allow this we have added logic to remove the last item of the Array List. We have added the appropriate getters to maintain encapsulation.

Hand:

Hand has attribute cardList to represent the cards in a hand. This method has the same addCard and removeCard methods as the Pack class, except with one notable difference. This difference is that when removing a card, you must remove it by index as the position you are removing cards from is not set (it will change depending on the cards a player picks up). We have added the appropriate getters to maintain encapsulation.

Player:

Player class represents the players in a game. Consequently, a player has an attribute “hand” to represent the four cards they hold. Secondly, they have “preference” which represents their player number and thus their card denomination. Finally, “Player” class also stores “drawDeck” and “discardDeck” attributes. These integer values represent the id of the decks the player will discard to and draw from. These attributes are needed to help with code readability and remove redundancy of calculation. Player has the methods, getHand(), getDiscardDeck(), getPreference(), getDrawDeck(), getDiscardDeck, draw(), discard() and hasWon(). Firstly, the getHand() method is needed to find which cards a player has from within card game using the player list. Secondly, getPreference(), is not only needed to check if the player is holding their preferred card, but it also acts as an ID which can be useful when comparing players for equality. Thirdly, getDrawDeck() and getDiscardDeck() are useful for reducing unneeded calculations of which deck has which id and instead provides direct access to the needed decks. Fourthly, draw() and discard() are methods which use a slew of other smaller methods inside other classes to encapsulate the process of drawing a card from the draw deck and discarding the card to the discard deck this simplifies the process by abstracting the problem into a class method making the main code more clear and concise. Finally, the hasWon() method is necessary to check if a player has winning cards currently in their hand.

CardGame:

This is the main script in which the game will run, bringing together all classes and objects. This is necessary as stated in the specification and therefore doesn't require much justification except that it is used to run main. In our Implementation CardGame has two class attributes. Firstly, we have an ArrayList called players (which contains all player objects) as information on players needs to be used at many points in the code, this implementation allows easy access to player objects. Secondly, for a similar reason, we have outputFiles. This is because many actions need to be logged for many different players, allowing global access to these files reduces the need to pass the files between methods every time a player takes any action and streamlines logical flow. CardGame contains the methods, createPack(), validatePack(), playerDiscards(), playerDraws(), hasPlayerWon(), logCurrentHand(), logDrawnCard(), logDisCard(), logGameEnd(), gameLoop() and endGame(). The first two methods (createPack() and validatePack()) are used to make sure the pack that has been input is a playable pack this helps to abstract the problem of if a pack is playable into two simple steps. Similarly, playerDiscards() and playerDraws() are used to abstract the process of a player drawing and discarding a card, this makes designing the threads much easier. Additionally, hasPlayerWon() also helps to abstract a part of the game loop within the threads making the code much more concise, readable and encapsulated. Furthermore, the log methods abstract the process of recording each of the players actions during multi-threading to allow for both easier understanding of who won and how and also make debugging easier. Finally, gameLoop() brings a few of the previous methods together into an atomic process where a player draws, discards and logs all their as one singular task.